Grafos

Estrutura de Dados

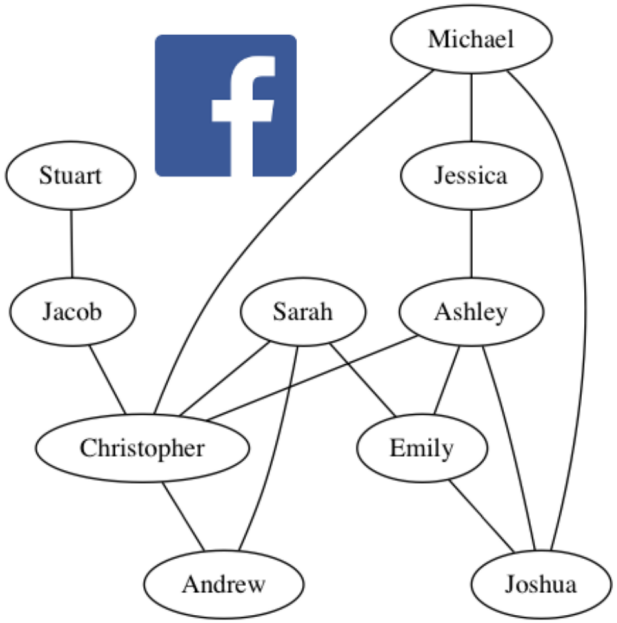
Universidade Federal de Juiz de Fora
Departamento de Ciência da Computação

Conteúdo

- Introdução
- Definição e conceitos de Grafos
- Representação de Grafos
- TAD
- Exercícios

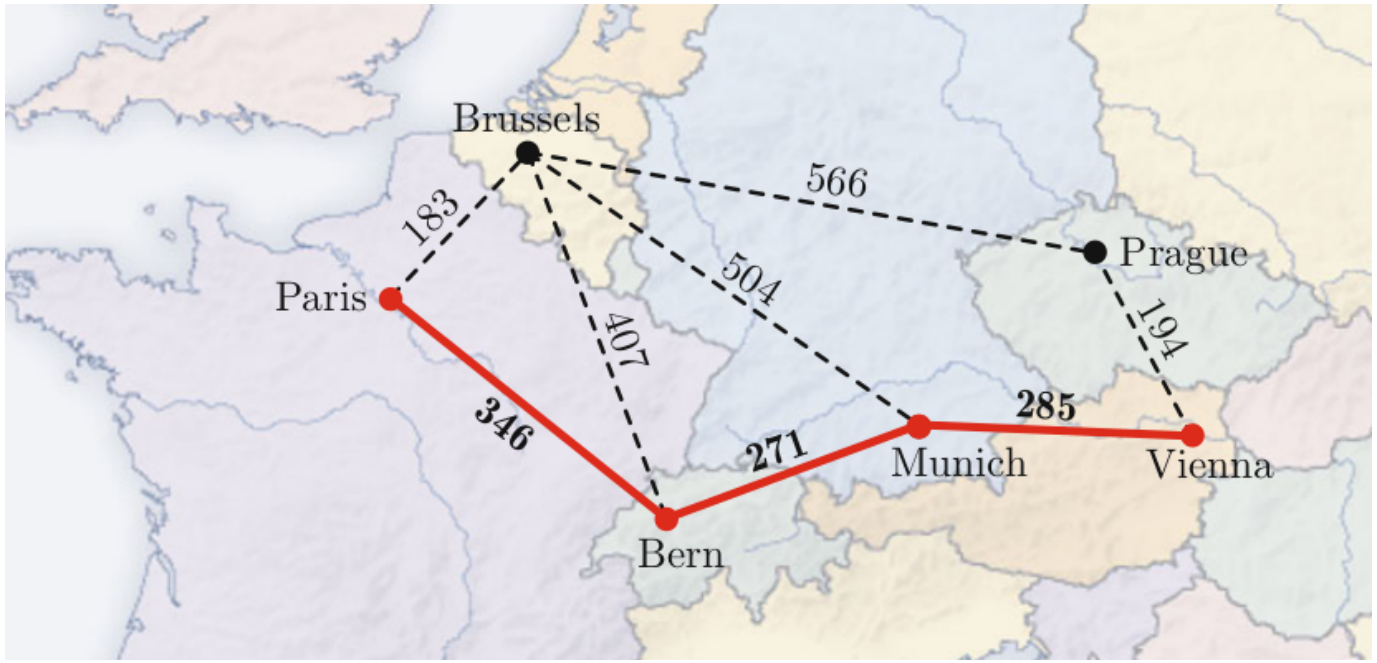
Introdução

- Grafos são uteis para representar várias estruturas que aparecem na prática.
- Exemplos:
  - Mapas
  - Relacionamentos (e.g. amigos no Facebook)



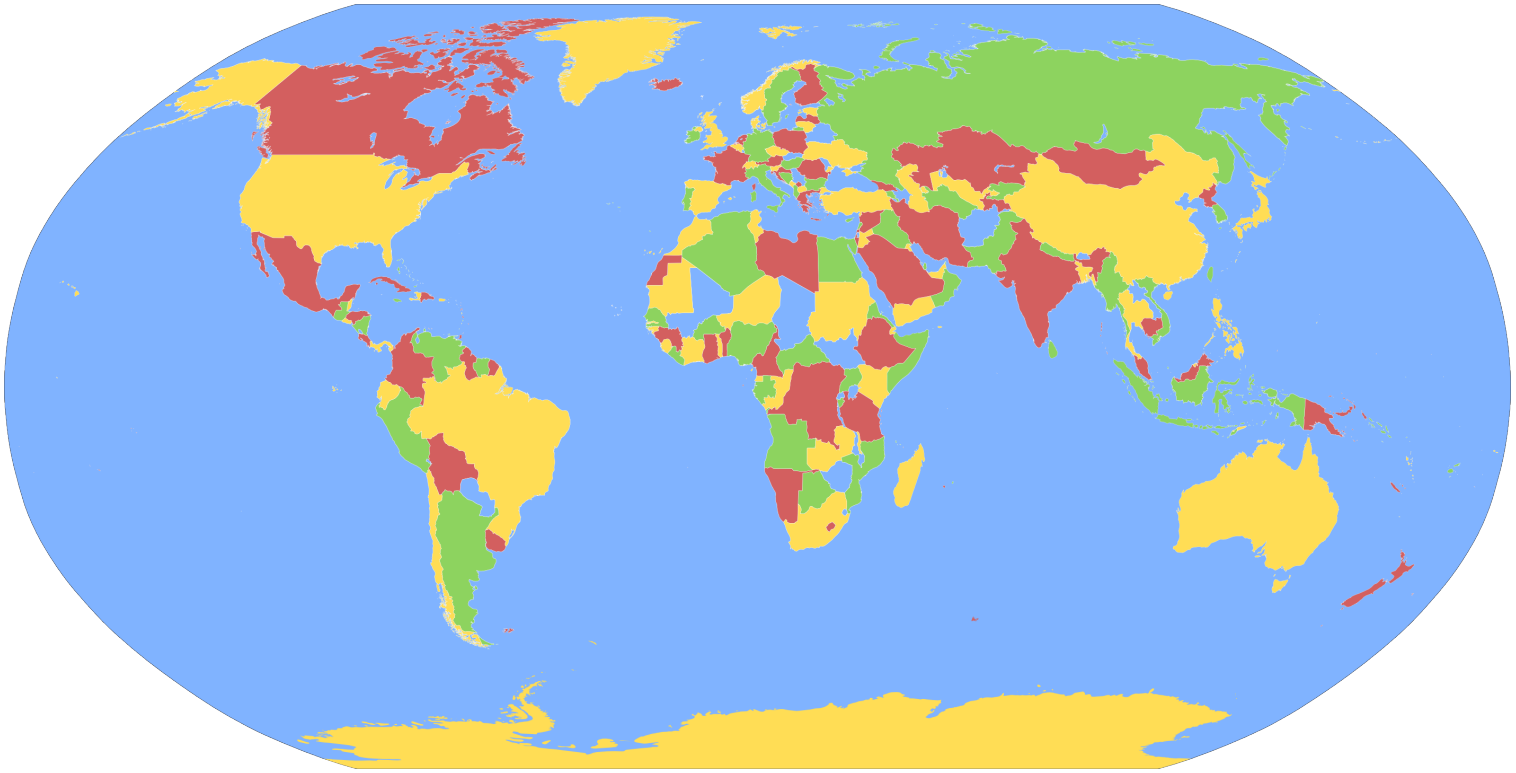
Aplicações

- Vários problemas de interesse podem ser modelados como um problema sobre um grafo.
  - Qual é o menor (melhor) caminho entre duas cidades?



Aplicações

- Vários problemas de interesse podem ser modelados como um problema sobre um grafo.
  - Problema de coloração de grafos (mapas)



Definição

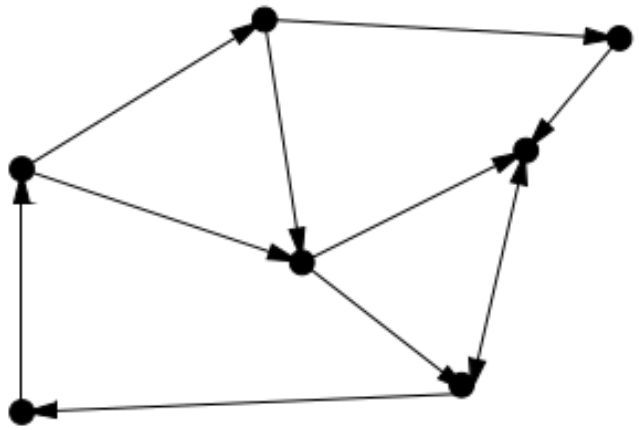
Grafo orientado

- Um grafo **orientado** é um par $G = (V, E)$ onde
  - V é um conjunto de elementos chamados **vértices**.
  - E é um conjunto de pares **ordenados** de vértices chamados de **arestas**.
  - Denota-se uma aresta por (u, v).

Definição

Grafo orientado

- Um grafo **orientado** é um par $G = (V, E)$ onde
  - V é um conjunto de elementos chamados **vértices**.
  - E é um conjunto de pares **ordenados** de vértices chamados de **arestas**.
  - Denota-se uma aresta por (u, v).
- Exemplo:



Definição

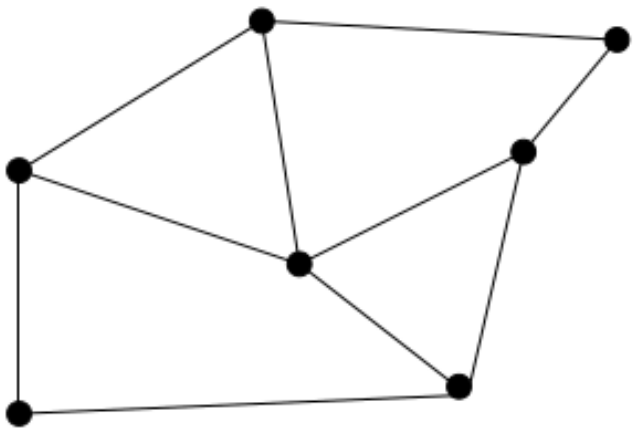
Grafo não-orientado

- Um grafo **não-orientado** é um par $G = (V, E)$ onde
  - V é um conjunto de elementos chamados **vértices**.
  - E é um conjunto de pares **não-ordenados** de vértices chamados de **arestas**.
  - Denota-se uma aresta por (u, v).
  - Note que neste caso $(u, v) = (v, u)$.

Definição

Grafo não-orientado

- Um grafo **não-orientado** é um par $G = (V, E)$ onde
  - V é um conjunto de elementos chamados **vértices**.
  - E é um conjunto de pares **não-ordenados** de vértices chamados de **arestas**.
  - Denota-se uma aresta por (u, v).
  - Note que neste caso $(u, v) = (v, u)$.
- Exemplo:



Grafos

- Os grafos **orientados** são mais genéricos do que os grafos **não-orientados**.
- Grafos não-orientados são mais convenientes quando a relação entre os vértices é **simétrica**.
- Nessa disciplina iremos estudar apenas a forma de representação (estrutura de dados - TAD) de grafos e alguns algoritmos simples.
- Aplicações, propriedades, algoritmos e análise serão estudados em **Teoria dos Grafos** e **Análise e Projeto de Algoritmos**.

Representação de Grafos

- As formas mais usadas para representar grafos são:
  - Matriz de Adjacência
  - Listas de Adjacências

Matriz de Adjacência

- Seja um grafo $G = (V, E)$ com n vértices e m arestas.
- A representação matricial é feita utilizando uma matriz M de tamanho $n \times n$, onde

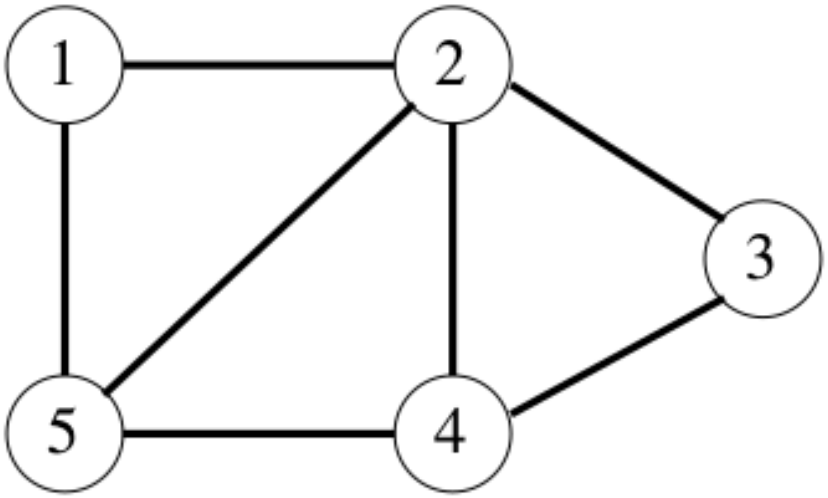
$$M[i, j] = \begin{cases} 1, & \text{se } i = j \\ 0, & \text{se } i \neq j. \end{cases}$$

- Esse tipo de representação permite respostas rápidas como: *a aresta* (i, j) *pertence ao grafo* G?
- Por outro lado, essa representação pode usar memória de forma excessiva para um grafo com muitos vértices e poucas arestas.

Matriz de Adjacência

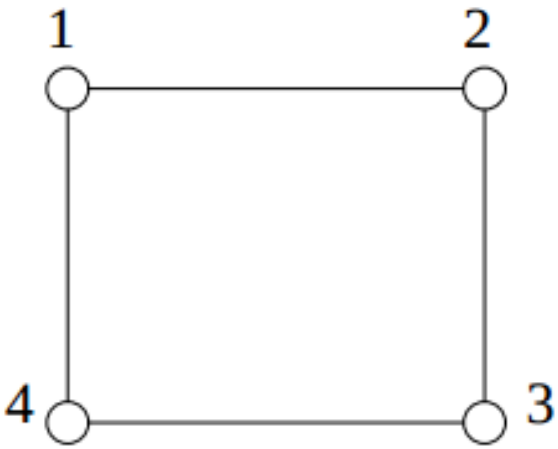
- Exemplo

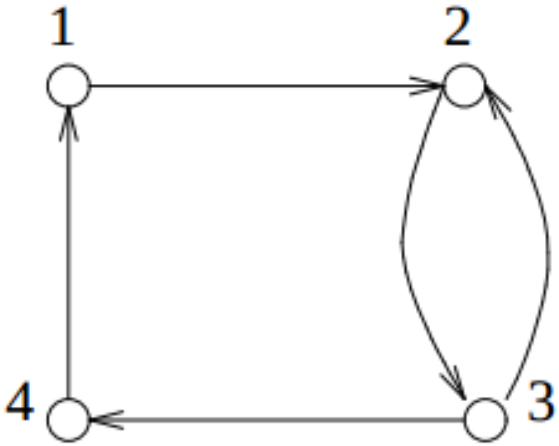
	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0



Matriz de Adjacência

- Qual a matriz de adjacência desses grafos?





Matriz de Adjacência

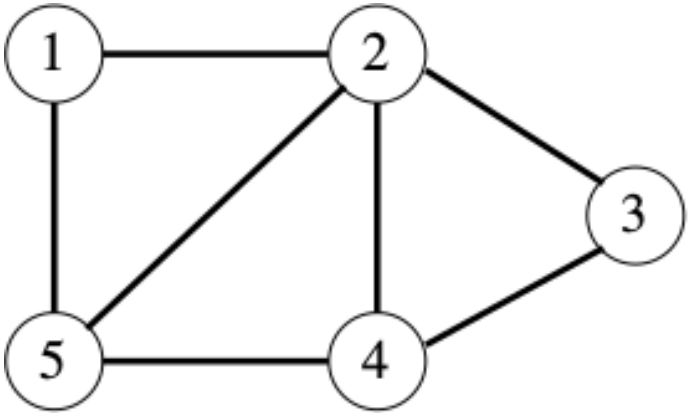
- Imagine um grafo que representa um mapa de uma cidade.
- Cada junção de duas ruas é um vértice do grafo.
- Junções vizinhas são conectadas por arestas.
- Considere uma cidade com 15 avenidas onde cada uma cruza com aproximadamente 200 ruas.
- Assim, tem-se 3000 vértices e 6000 arestas.
- Matriz de adjacência:
  - $3000 \times 3000 = 9000000$ elementos na matriz
  - A maioria dos elementos é vazio (com valor igual a 0)!

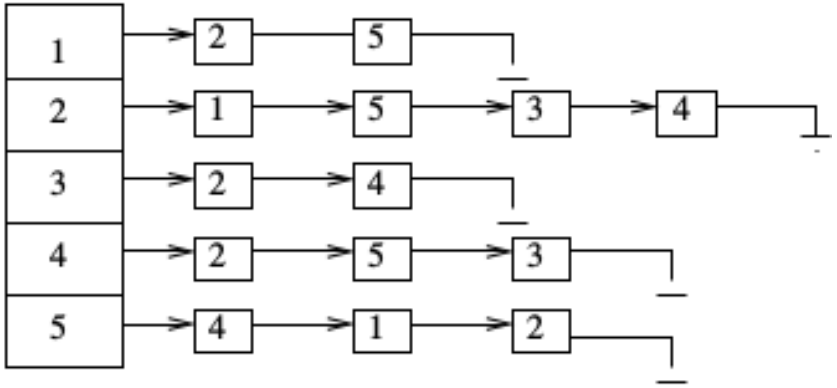
Lista de Adjacência

- Pode-se representar um grafo usando uma **lista encadeada** para armazenar os vizinhos adjacentes a cada vértice do grafo.
- Em uma lista de adjacência é mais difícil verificar se uma dada aresta (i, j) está no grafo G, já que é preciso procurar na lista apropriada para encontrar a aresta.
- A representação com listas de adjacência é a mais adequada para a maioria das aplicações com grafos.
- Vamos estudar como representar grafos usando **lista de adjacência**.

Lista de Adjacência

- Exemplo





TAD Grafo

- Para representar grafos, vamos manter uma contagem de quantos vértices cada grafo tem e, ainda, associar a cada vértice uma identificação única de 0 até $nVertices - 1$.
- Assim, cada vértice possui `info`, ponteiro para o próximo vértice `prox` (e, pode ainda, incluir um `peso`).

```
class No
{
public:
    No()                        { }
    ~No()                       { }
    int consultaInfo()          { return info; };
    No * consultaProx()         { return prox; }
    void atribInfo(int val)     { info = val;  }
    void atribProx(No * p)      { prox = p;    }
private:
    int info;     // informacao de adjacencia
    No * prox;    // ponteiro para a prox na lista
};
```

TAD Grafo

```
#define MAXV 1000

class Grafo
{
public:
  Grafo(bool orient);
  ~Grafo();
  void leGrafo();
  void insereAresta(int x, int y, bool orient);
  void imprime();

private:
  No *arestas[MAXV];
  int nVertices;
  int nArestas;
  bool orientado;
};
```

TAD Grafo

```
Grafo::Grafo(bool orient)
{
    nVertices = 0;
    nArestas  = 0;
    orientado = orient;

    for(int i=0; i<MAXV; i++)
        arestas[i] = NULL;

    // monta o grafo
    leGrafo();
}
```

TAD Grafo

```
void Grafo::leGrafo()
{
    int m,n,x,y;

    cout << "Num de vertices e arestas" << endl;
    cin >> n >> m;

    nVertices = n;
    nArestas  = m;

    for(int i=0; i<m; i++)
    {
        cout << "Digite aresta: " << endl;
        cin >> x >> y;
        insereAresta(x,y,orientado);
    }
}
```

TAD Grafo

```
void Grafo::insereAresta(int x, int y, bool orient)
{
    if (x >= 0 && x < nArestas &&
        y >= 0 && y < nArestas)
    {
        No *p = new No();
        p->atribInfo(y);
        p->atribProx(arestas[x]);

        arestas[x] = p;

        if (orient == false)
            insereAresta(y, x, true);
        else
            nArestas++;
    }
}
```

TAD Grafo

```
void Grafo::imprime()
{
    No *p;

    cout << "V -> E" << endl;
    for(int i=0; i<nVertices; i++)
    {
        cout << i << " ->";
        p = arestas[i];
        while(p != NULL)
        {
            cout << " " << p->consultaInfo();
            p = p->consultaProx();
        }
        cout << endl;
    }
}
```

Percursos em Grafos

- Existem difentes formas de percorrer um grafo
- Percurso em Largura
  - Usa uma fila como estrutura de dados auxiliar
- Percurso em Profundidade
  - Usa uma pilha como estrutura de dados auxiliar
  - Se implementado com recursivadade, elimina a necessidade de uma pilha de forma explícita.

Percursos em Grafos

- Percurso em Largura
- *Breadth-first search (BFS)*

```
procedure BFS(G,v) is
    let Q be a queue
    Q.enqueue(v)
    label v as discovered
    while Q is not empty
        v = Q.dequeue()
        process(v)
        for each w in Adj(v) do
            if w is not labeled as discovered
                Q.enqueue(w)
                label w as discovered
```

TAD Grafo

```
void Grafo::percursoLargura(int s) {
    int v, y;
    bool encontrado[MAXV];
    for(int i=0; i<nVertices; i++) encontrado[i] = false;

    No * p;
    Fila * f = new Fila(MAXV);
    f->entra(s);
    encontrado[s] = true;
    while(!f->vazia()) {
        v = f->consultaInicio();
        f->sai();
        cout << "vertice " << v << " processado\n";
        p = arestas[v];
        while(p != NULL) {
            y = p->consultaInfo();
            if(encontrado[y] == false){
                f->entra(y);
                encontrado[y] = true;
            }
            p = p->consultaProx();
        }
    }
    delete f;
}
```

Percursos em Grafos

- Percurso em Profundidade com implementação recursiva
- *Depth-first search (DFS)*

```
procedure DFS(G,v):
    label v as discovered
    process(v)
    for each w in Adj(v) do
        if vertex w is not discovered then
            DFS(G,w)
```

Percursos em Grafos

- Percurso em Profundidade com implementação iterativa
- Utiliza uma pilha de forma explícita

```
procedure DFS-iterative(G,v):
    let S be a stack
    S.push(v)
    while S is not empty
        v = S.pop()
        process(v)
        if v is not labeled as discovered:
            label v as discovered
            for each w in Adj(v) do
                S.push(w)
```

TAD Grafo

```
void Grafo::percursoProfundidade(int s)
{
    bool encontrado[MAXV];
    for(int i=0; i<nVertices; i++) encontrado[i] = false;

    percursoProfundidadeAux(s, encontrado);
}

void Grafo::percursoProfundidadeAux(int s, bool * encontrado)
{
    int v, y;
    No * p;

    encontrado[s] = true;
    cout << "vertice " << s << " processado" << endl;

    p = arestas[s];
    while(p != NULL) {
        y = p->consultaInfo();
        if(encontrado[y] == false)
            percursoProfundidadeAux(y, encontrado);

        p = p->consultaProx();
    }
}
```

Exercícios

1. Incluir no TAD `Grafo` a informação sobre o grau de cada vértice.
2. Modifique o algoritmo de percurso em largura para contar quantas arestas foram encontradas no grafo. (Considere que depois de sua criação, novas arestas podem ser inseridas, logo o número de arestas `nArestas` pode ter sido alterado).
3. Implemente o percurso em profundidade utilizando de forma explícita uma pilha ao invés de realizar a chamada recursiva.